Future Generation Computer Systems 174 (2026) 107969



Contents lists available at ScienceDirect

Future Generation Computer Systems

journal homepage: www.elsevier.com/locate/fgcs





Blockchain-integrated storage and forwarding system

Zhijie Han [a],*, XingBo Xie [a], Xiaoyu Du [b], Ying Du [b], Xin He [a]

[a] School of Software, Henan University, Kaifeng, 475000, China
[b] School of Computer and Information Engineering, Henan University, Kaifeng, 475000, China

ARTICLE INFO

Keywords:
BlockChain
Merkle hash tree
Data storage
Data integrity
Data forwarding

ABSTRACT

With the widespread use of cloud computing technologies, cloud storage services face significant hurdles in terms of data security and rapid retrieval. Centrally stored data is prone to modification and unlawful access, making data integrity and traceability a critical concern. In this research, we present a blockchain-based integrity verification scheme for cloud data storage and transport. Merkle and interval trees are utilized to achieve efficient data indexing, while the Elliptic Curve Integrated Encryption (ECIES) technique ensures the security of data proofs. It also incorporates the Kademlia algorithm to optimize the data backup and transmission path, as well as the transmission path voucher chain (TPV) to record each node's transmission contribution, ensuring data transmission integrity and traceability. Experimental results reveal that the technique considerably enhances data storage security and retrieval efficiency in a cloud setting.

1. Introduction

In the information era, the Internet has become an essential component of modern life, and its rapid expansion has resulted in a significant increase in the number of Internet users. According to the most recent figures, the number of Internet users worldwide has surpassed billions, and this figure continues to rise. According to the Statistical Report on Internet Development in China, as of June 2023, the number of Internet users in China reached 1.079 billion, an increase of 11.09 million compared with December 2022. The Internet penetration rate reached 76.4%, an increase of 0.8 percentage points compared with December 2022 [1]. Traditional cloud storage [2] systems are convenient for storing and accessing data, but they have inherent problems. For example, a centralized storage design is vulnerable to a single point of failure, and if a central server fails, all data that relies on it is jeopardized. Furthermore, there is a rising lack of consumer trust in cloud service providers, with users concerned about the privacy and security of their data. The risk of data destruction, loss, or unauthorized access drives customers to seek more reliability and security from cloud storage.

In order to address these issues, blockchain technology has arisen as a new method of storing and sending data. Blockchain technology, first introduced by Satoshi Nakamoto in 2008 [3], has gained attention from numerous areas due to extensive research. Blockchain's decentralized structure, immutability, and openness make it perfect for ensuring data security and privacy. Blockchain technology offers a safe and dependable method for data storage and distribution.

However, determining how to efficiently verify and transmit data on a blockchain network has become a critical issue in the development of blockchain applications. Juan et al. [4] presented a blockchain storage architecture using the InterPlanetary File System (IPFS). IPFS maintains data integrity and untampering through content addressing and Merkle tree structure, while employing DHT to enable quick node discovery and data retrieval, however, its reliance on node voluntary participation may result in data persistence issues and a lack of economic value. Protocol Labs [5] started the Filecoin project. Filecoin integrates IPFS content addressing with blockchain technology. Filecoin ensures data integrity and availability through Proof-of-Replication and Proof-of-Space, and it offers economic incentives for node participation; however, the system's complexity can lead to operational inefficiencies, and its storage and retrieval costs can be influenced by market fluctuations. The Storj platform was built by Shawn Wilkinson and his team [6]. Storj uses end-to-end encryption and data sharding to protect data integrity and reliability with redundant storage strategies, as well as Kademlia-like algorithms to optimize node discovery, but the level of privacy protection is dependent on the storage provider's honesty and may be vulnerable to a single point of failure. The Swarm system was developed by Viktor Trón and colleagues [7]. Swarm uses Ethereum's smart contracts and the Kademlia algorithm. Swarm preserves data integrity and allows for efficient data transfer over a P2P network, although performance and cost are affected by the status of the Ethernet network, particularly when the network is congested, and rising Gas prices may raise the cost of storing and retrieving data.

In conclusion, the primary focus of our research is on two areas: data integrity verification and data transfer. The suggested strategy in this

* Corresponding author.
E-mail address: hanzhijie@126.com (Z. Han).

https://doi.org/10.1016/j.future.2025.107969
Received 8 May 2024; Received in revised form 31 May 2025; Accepted 5 June 2025
Available online 23 June 2025
0167-739X/© 2025 Published by Elsevier B.V.

work is based on blockchain technology, which allows for secure data transfer and data integrity via the interaction of the CSP, DO, and Other nodes. The major contributions of this study are mentioned below.

- We create a blockchain-enabled store-verify-and-forward architecture, BISF, that allows data owners to execute data integrity checks at any time. Meanwhile, data retrieval uses the Kademlia algorithm to optimize the transmission rate, increasing the network's overall data transmission efficiency.
- We designed a new verification method. Combining the methods of data chunking, Merkle tree, and interval tree, by chunking the data and constructing a Merkle tree, the root hash of the Merkle tree is used in conjunction with the efficient retrieval mechanism of the interval tree to improve data integrity verification efficiency and speed in a distributed storage system.
- The technique makes use of the Kademlia algorithm for data backup and transfer in order to increase network transmission rate. To secure data integrity during transfer, we create a transmission credential mechanism. This approach not only protects the data's safety and security, but it also gives explicit proof of involvement for each node in the transmission.

The organization of the remaining parts of this article is as follows: Section 2 introduces the state of research in this field; Section 3 presents the relevant concepts of this work; Section 4 describes the storage framework proposed in this study; Section 5 details the forwarding framework developed through this research; Section 6 analyzes the security aspects of this work; Section 7 discusses the application of our framework to real-world scenarios; Section 8 discusses the experimental results and performance analysis of this study; Section 9 concludes the findings of this work.

2. Related work

2.1. Blockchain-based data integrity verification

Liu et al. [8] proposed a regenerative code based public auditing protocol for cloud storage privacy protection. The protocol consists of three steps: setup, auditing, and fixing, where in the setup step, the data owner generates key pairs, encryption keys to distribute to the agent, and generates authentication for file blocks, and finally stores the authentication and encoded blocks in the cloud server. Subsequently, Zhang et al. [9] pointed out the loopholes existing in this mechanism and conducted an analysis. Xue et al. [10] offered a decentralized cloud storage system with encryption, smart contracts, and Merkle trees. The system conducts regular audits of data integrity using Merkle trees and employs an incentive mechanism to ensure data availability. Meanwhile, optimizing storage resources lowers the cost. Yue et al. [11] offered a MerkleTree-based verification technique, examined MHT performance under various architectures, and proposed a fair sampling strategy to improve sampling verification effectiveness. Liu et al. [12] proposed a data integrity auditing technique that uses the quadranomial Merkle tree and blockchain to increase efficiency by minimizing the number of intermediate nodes and hashing processes. Use smart contracts to automate the verification process and decrease the need for manual involvement. Quadtree improves storage and computing efficiency, but it may increase search complexity. He et al. [13] presented a new blockchain storage structure based on the T-Merkle tree, intending to optimize storage efficiency and achieve rapid binary search. By using ZSS short signature technology to generate data labels, the security and integrity of data are improved. Li et al. [14] presented a novel blockchain-based auditing method, which intends to improve the transparency of the auditing process and minimize the requirement for third-party audits by uploading data block tags to the blockchain. Increase openness in the auditing process and reduce reliance on and cost of third-party audits. However, when dealing with huge amounts of data, uploading all of the tags to the blockchain may have a detrimental impact on the blockchain's performance, reducing auditing efficiency. Chen et al. [15] presented a blockchain-based dynamic and traceable data integrity verification scheme, primarily using the homomorphic verifiable labeling technique and the linear probing Merkle hash tree (LP-MHT) structure. The LP-MHT collects data tags from numerous smart devices to achieve dynamic data processing and storage optimization, allowing the system to handle data updates and verification more efficiently. Wang et al. [16] proposed a blockchain-based decentralized data outsourcing auditing scheme. The scheme leverages the decentralized nature of blockchain and smart contracts to allow users to securely store data and perform integrity audits without relying on a specific TPA or administrator. Through smart contracts, users interact with audit service providers and cloud service providers to ensure data integrity and privacy protection. Tu et al. [17] proposed an improved multi-copy cloud data auditing scheme, which consists of the following steps: the user generates key pairs, creates tags and IMB trees, and then sends the data and IMB trees to the cloud service provider. When auditing is required, a trusted third-party auditor (TTP) launches an audit challenge to the cloud service provider, which generates evidence based on the challenge and returns it to the TTP, which verifies the correctness of the evidence.

2.2. Data transmission

Zhang et al. [18] presented a unique IoT data-transmission system that uses blockchain technology and edge computing to improve data security and dependability. Data transmission security is further enhanced by constructing a three-layer architecture consisting of an IoT layer, an abstract IoT layer, and a blockchain layer, as well as a four-step process (device registration, message transmission, bookkeeping node election, and authentication rewards), as well as evaluating participants' credit via a credit metric mechanism. Yu et al. [19] presented a scalable blockchain P2P network transmission model that improves data transfer by adding transmission lines and separating the K-bucket into many sub-regions. The model enhances transmission efficiency and security by filtering nodes that have already received data to avoid repetitive forwarding, while also ensuring that data is routed to the target node via numerous redundant nearby nodes. Wu et al. [20] provided MapChain-D, a distributed blockchain architecture for large-scale industrial IoT network design. The system employs a dual blockchain topology that maps the data chain to the index chain, hence improving data storage and retrieval efficiency. It is implemented using Kademlia's DHT and PoA consensus method, which are intended to reduce storage and communication overheads while improving data security and dependability. Yuan et al. [21] offered a trusted data sharing system named TRUCON. TRUCON optimizes data forwarding using the Kademlia algorithm and manages channel congestion by altering bucket size and distance thresholds. Meanwhile, a method for deduplicating and identifying traffic data using a cuckoo filter is developed to reduce duplicate sharing. In addition, a blockchain trust management method with both full and light nodes is proposed. Shen et al. [22] offered a SEKad protocol, through optimizing blockchain network broadcast efficiency and scalability through geographic information clustering and improved Kademlia network topology, as well as reducing data transmission redundancy by utilizing the minimum subtree and minimum redundancy transmission mechanisms, have been experimentally proven to significantly improve network performance.

3. Preliminaries

In this section, we provide preliminary knowledge used in this paper, including ECIES, Interval Tree, and Kademlia Algorithm.

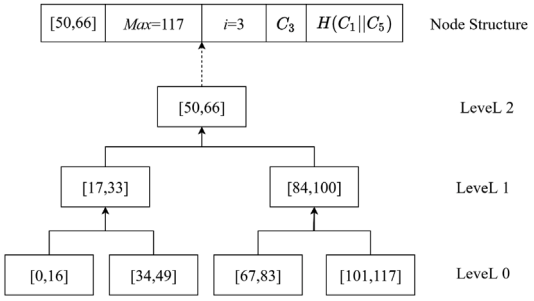


Fig. 1. The structure of Interval Merkle tree.

3.1. Elliptic curve integrated encryption scheme

ECIES (Elliptic Curve Integrated Encryption Scheme) [23] is a hybrid encryption system based on elliptic curve cryptography that incorporates the advantages of both symmetric and asymmetric encryption.

Encryption: In ECIES encryption, the sender first generates a temporary elliptic curve private key r and computes the corresponding temporary public key R. Next, the receiver's public key is used to generate, via the Elliptic Curve Diffie–Hellman (ECDH) protocol, the shared key S. The shared key S is then processed by the Key Derivation Function (KDF), which produces the symmetric encryption key K_{enc} and the messageauthentication code key K_{mac}. The plaintext message M is encrypted into a ciphertext C using K_{enc}, and the message authentication code (MAC) of C is generated using K_{mac} to ensure the integrity of the ciphertext. Finally, the sender sends the temporary public key R, the ciphertext C, and the MAC to the receiver.

In the ECIES decryption process, the receiver uses its own private key SK to compute the same shared key S as that of the sender via the received temporary public key R. The shared key S is then used to generate the symmetric encryption key K_{enc} and the message authentication code key K_{mac} via the same key derivation function. The receiver uses K_{mac} to authenticate the received message authentication code to ensure that the ciphertext has not been tampered with during transmission. After the verification, K_{enc} is used to decrypt the ciphertext C to recover the original plaintext message M. This process ensures that the security and integrity of the message is effectively protected.

3.2. Interval tree

The Interval Tree [24] is based on the Red-Black Tree, which processes and stores interval data. In the interval tree, each node represents an interval, and its key value is the interval's left endpoint. In addition to an interval, each node may hold additional information about the interval, such as its maximum value. This enables the interval tree to carry out lookup, insertion, and deletion operations in O(log n).

To improve verification efficiency and reduce online storage pressure, the Interval tree is modified by (1) storing values at nodes as committed values of the root hash of the Merkle hash tree instead of hashes of individual data fragments; (2) inserting labeled indexed intervals into the nodes; (3) attaching the indexed maximal finger of the subtree in which each node is located to each label, while the indexes of the individual intervals are recorded in nodes. The tree structure is shown in Fig. 1.

3.3. Kademlia algorithm

Kademlia [25] is the result of a study published in 2002 by Petar Maymounkov and David Mazières of New York University, U.S.A. The Kademlia protocol maps a node's position in the network to a k-bit binary ID space (typically 160 bit) based on the unique identifier of the message (usually a hash of the data content), with each participating

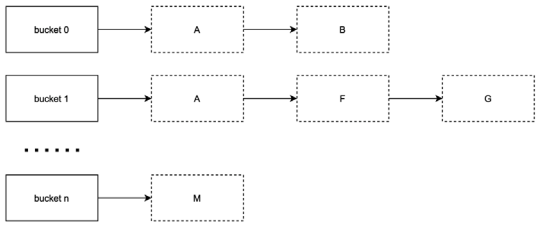


Fig. 2. k-bucket structure.

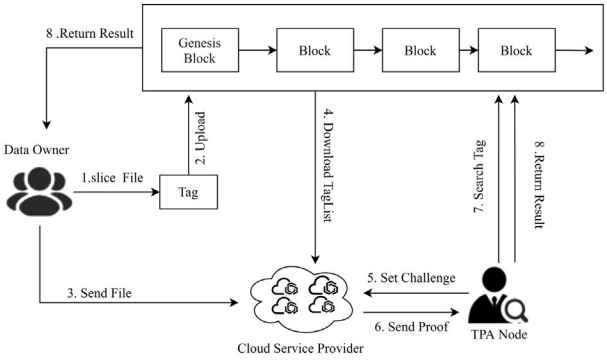


Fig. 3. The proposed store system model.

node having a unique ID. The core goal of the protocol is to quickly locate and find nodes that have a particular ID or that store the corresponding data. Its main feature is to measure the "distance" between nodes through the XOR algorithm, where the closer the distance is, the more similar the IDs are, to achieve an efficient lookup. Each node maintains a set of contact lists (Bucket), these lists are divided into levels according to the distance of node IDs, and each bucket contains a certain number of nearest-neighbor nodes. When a new node joins or an existing node goes offline, the network is able to adjust itself quickly to maintain stability and accessibility. As illustrated in Fig. 2, each network node establishes a dedicated routing table structure, termed a 'k-bucket', to retain vital distance-related information.

4. Storage model design

This section primarily explores the storage framework of the BISF scheme along with its detailed steps and discusses the design of a data integrity verification strategy within this framework. The notational definitions employed throughout this paper are presented in Table 1.

4.1. Model fram ework

Our proposed model for blockchain-based data integrity verification is depicted in Fig. 3. It principally comprises four key entities: Data Owner(DO), Cloud Storage Provider(CSP), the Blockchain(BC) and TPA Node. These entities are defined as follows:

(1) DO: Seeks to upload data to the CSP and may also function as a Trusted Third Party Auditor.
(2) CSP: Charged with the storage of DO's data and the provision of corresponding integrity verification.
(3) BC: Used to record information about DO and CSP interactions.
(4) TPA Node: Any node user, cloud server provider, data consumer, or another third party in the network can operate as a TPA.

The framework supports data integrity and verifiability by data segmentation and makes use of Merkle trees and blockchain technology. The user's data is separated into different sections and organized into

Table 1
Notations for the system model.

Notations	Description
ID_i	DO's identification
PK_i, SK_i	DO's public and private keys
F	File to be uploaded
T_i	Secondary label
β	The list of Secondary label
Index	The index of File fragment
UID_i	Identifier of the File block

groups, and each group is used to generate root hashes by building a Merkle tree, which are then uploaded to the blockchain as tags alongside other metadata to make a traceable interval tree. When an auditor conducts a data interrogation, the cloud storage service provider can swiftly locate and extract the associated data block using the index provided, resulting in a Data Proof(DP) of ownership. To improve data security while in transit, the system encrypts the DP of Ownership with ECIES. Auditors then evaluate the submitted proofs to ensure the data's authenticity and integrity, and the validation findings are publicly broadcast so that all relevant parties may monitor the status of data storage and auditing in real time.

4.2. Specific scheme

4.2.1. Initialization phase

The DO divides the file F equally into equal-sized blocks, represented as F_1, F_2, …, F_n, where n is the total number of blocks and i is the number of file blocks in each group.

$$F = (F_1, \ldots, F_i), \ldots, (F_m, \ldots, F_{m+i}), \ldots, (F_{n-i}, \ldots, F_n) \tag{1}$$

A hash value is computed for each data block F_i to generate a data tag T_i. The data tag serves as an identifier of the file block for subsequent integrity verification.

$$T_i = SHA256(F_i), 1 \le i \le n \tag{2}$$

The collection of data labels β is: $\beta = T_1, T_2, \ldots, T_i$.

4.2.2. Storage phase

Upon receiving the data block set F and the file label set β, the storage provider recalculates the hash value for each data block F_i and compares it with the provided label set β to verify that the data is intact and has not been tampered with during transmission. And then the data block labels β are grouped and each group contains m labels. A Merkle hash tree is constructed for each group of data blocks to generate a group-level root hash R_j. The relevant Merkle tree is then created after the file has been encrypted. Create a Merkle tree for each set of data blocks. Where m represents the group's initial index.

$$R_j = MHT(T_1, T_2, \ldots, T_i), 1 \le j \le \frac{n}{i} \tag{3}$$

After MHT construction, the corresponding commitment value data labels C_j are then generated using the Pedersen commitment pair R_j.

$$C_j = g^{R_j} h^r mod p \tag{4}$$

The storage chamber associates the index range $[min_j, max_j]$ of each group of data blocks with the corresponding group-level root hash data label C_j, constructs an interval Merkle tree and uploads it into the blockchain to ensure the immutability of the stored data and labels.

4.2.3. Audit phase

DO can challenge the CSP against a file block, by its own pk, UID and Index.

$$Chal = ChalGen(PK, UID_i, Index) \tag{5}$$

The service provider quickly locates the interval in which the corresponding data block is located through the interval Merkle tree based on the challenge and finds the corresponding C_j and extracts the MHT path proof for the data block containing: all the hashes T_{chal} of the data block F_{chal} and all the sibling node hashes on the path from the leaf node to the root node. The repository combines the path proof with Q to generate the storage Proof.

$$Q = \sum_{i=1}^{n} |C_i \, Chal_i \, mod \, p| \tag{6}$$

$$Proof = Q, r, MerklePath \tag{7}$$

The repository encrypts the Proof of Storage using the ECIES encryption scheme to generate the ciphertext CM. Computes the temporary public key R and the shared key S. Derives the symmetric key K and the message authentication code key K_{MAC} from S using the key derivation function. The final ciphertext C_k is $C_k = \{R, CM, MAC\}$.

4.2.4. Verify phase

The verifier decrypts the ciphertext C_k using the private key SK and computes the shared key S. The symmetric key K and the message authentication code key K_{MAC} are derived from S using the key derivation function. The MAC is verified to ensure message integrity. The Proof of Storage Proof is then decrypted. The verifier reconstructs the MHT group level root hash $\dot{R}_j$. If $\dot{MAC} = MAC$, then the message has not been tampered with. The result P of the stored proof is then verified to determine whether P is consistent with Q. The MHT group-level root hash $\dot{R}_j$ is reconstructed. The algorithm for the verification process is shown in Algorithm 1.

$$P = \sum (|g^{\dot{R}_j} h^r \, mod \, p|) \, Chal_j \, mod \, p \tag{8}$$

Algorithm 1 Validation Process Algorithms

Input: pk, UID_i, Index, Blockchain root hash $Root_k$
Output: Verification result 'isEqual'

```
Chal ← ChalGen(PK, UID_i, Index)
DP ← GenProof(Index)
encryptedDP ← Encrypt(DP)
send {encrypted DP, R, authTag} to TPA
VerifyResult ← VerifyProof(Decrypt(encryptedDP))
if VerifyResult == DP ∧ authTag == authTag then
    isEqual =True
    broadcast "Data ownership verified"
else
    isEqual =False
    broadcast "Data ownership verification failed"
end if
return isEqual
```

5. Forward model design

This section primarily introduces the forwarding framework of the BISF scheme along with its specific steps, and outlines the design of a transmission credential strategy within this framework. The notational definitions employed throughout this paper are presented in Table 2.

5.1. Model frame work

The architectural diagram of the blockchain-based data forwarding mechanism is shown in Fig. 4. Five distinct role entities are outlined: Data Retriever, CSP, Data Forwarder, Blockchain and Node. The functions of these entities are as follows:

(1) DR: The node responsible for initiating the retrieval request.
(2) DF: The node that facilitates the forwarding of data.
(3) Node:The node that stores the corresponding retrieved data.

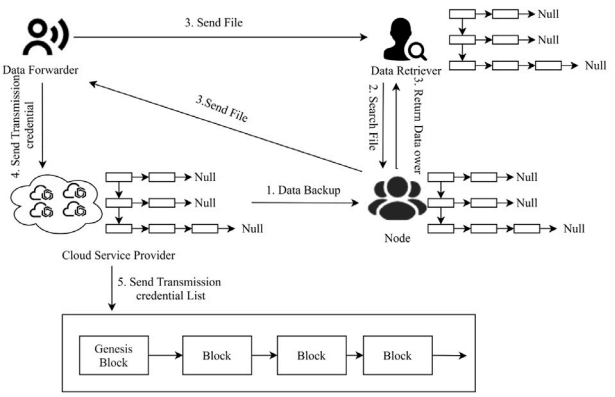


Fig. 4. The proposed forward system model.

Table 2
Notations for the system model.

Notation	Description
TimeStamp	Timestamp
TC	Transmission credential
SID	Source ID
NID	Next hop node ID
FHash	File Hash
PreHash	Hash of the previous TC
CID	Current node ID

This technique makes use of the Kademlia algorithm to achieve distributed storage and rapid data retrieval. The user provides the data to the CSP, which computes the hash value for each data slice. The CSP then distributes the data slices across the Kademlia network to the node closest to the XOR of the data slice's hash value for storage and redundant backup. This procedure provides a proper distribution of data slices in the Kademlia network while also improving data retrieval performance and reliability.

To assure data retrieval accuracy and transmission transparency, we present a new data transmission method based on the Kademlia algorithm and chain structure. When data is transported across the Kademlia network, each participating node creates data transmission credentials(TC) to record transmission channels and node contributions, which are then sent to the CSP to create a Transmission Path Voucher Chain(TPV). This approach not only ensures data integrity, but it also improves traceability and transparency of the transmission process by tracking each node's contribution in real time via the TPV, resulting in a reliable and transparent record system for data transmission.

5.2. Transmission path voucher chain

We use a chain structure to improve the integrity and traceability of data transmission. A Transmission Path Voucher Chain is a chain structure that assures data integrity by documenting every essential step in the data transmission process and providing proof of participation for each node involved. The construction procedure is depicted below. The algorithm for generating and verifying the transmission chain can be found in Algorithm 2.

(1) Transmission credential generation: when a data fragment is transmitted, the transmitting node creates a transmission credential that includes FHash, UID_i, SID, CID, NID, TimeStamp, and PreHash, digitally signs it, and sends it to the CSP, which serves as the foundation for constructing the TPV.

(2) TPV Construction: After generating the credentials, the node passes the source data and TimeStamp to the next node. After receiving the source data and timestamp from the source node, it calculates the source node's credential hash value and produces its own credentials. Each receiving node follows the processes above to calculate the previous node's hash value and generate its own credentials.

(3) Data Integrity Verification: When data integrity needs to be checked, the CSP uses TPV to evaluate each segment of data transmission. It can begin at any point in the data transmission process, trace the complete path, and validate the integrity and authenticity of the data segments by comparing the consistency of the data hash value to the credential hash. This technique ensures that the data is not tampered with during transmission, hence ensuring data security and reliability.

Algorithm 2 TPV Construction and Integrity Validation

Input: File hash, Data identifier, Source node ID, Target node ID, Current ID, Timestamp, Previous voucher hash
Output: Transmission Credential Confirmation

for each node n_j in transmission path **do**
  if n_j == SID **then**
    Compute file hash
    Create TC_i = sign(FHash, UID_i, SID, CID, NID, TimeStamp)
    Send TC and File to next node
  else
    Compute previous TC hash h_{prev}
    Compute Current TC
    Send TC and File to next node
  end if
end for
{Data Integrity Validation}
for each TC_i in TPV **do**
  Compute current TC_i
  if $TC_i.PreHash \neq TC_{i-1}$ **then**
    return "Data has been tampered"
  end if
  if $TC_i.dataHash \neq FHash \wedge TC_i.dataHash \neq TC_{i-1}.dataHash$ **then**
    return "Data integrity failed"
  end if
end for

6. Security analysis

This section primarily analyzes the security of the BISF system, demonstrating that our proposed system meets the required security attributes.

6.1. Data integrity analysis

After receiving the encrypted version of the data block F_i and the Merkle path information, the verifier computes the root hash value. Due to the unidirectional and collision-resistant nature of the hash function, small changes can result in significant changes in the hash value. Therefore, the new hash value is different from the original hash value, and if the hash value of a leaf node is changed, it will affect the hash value of its parent node all the way to the root node, and any change in the hash value of any node will affect the computation of the nodes at the upper level. Since the root hash depends on the entire tree structure, the root hash will change even if only the hash of a single data block is changed. This means that after tampering with a block of data, the root hash must be different from the original.

In Pedersen commitment, the commitment value C_j is computed based on the root hash value R_j and the random factor r.

$$\dot{C_j} = g^{\dot{R_j}}\, h^r \bmod p \tag{9}$$

Compare with Formula (4), since $g^{R_j} \neq g^{\dot{R_j}}$, so the commitment values cannot be the same. The variation of the root hash value directly

affects the calculation of the Pedersen commitment even when the parameters g, h, p are known. Pedersen commitment is based on the large number problem with unpredictability and unforgeability, which ensures that the correct commitment value cannot be forged after tampering with the data, leading to different calculation results.

6.2. Privacy

This scheme ensures the reliability of the data transmission process by using ECIES encryption and MAC. The computational complexity that relies on the Elliptic Curve Discrete Logarithmic Problem (ECDLP) ensures that it is difficult to crack with the current computational power. Its cracking difficulty is $O(2^{n/2})$, providing an extremely high level of security. Two symmetric keys K_{enc} and K_{mac} are used, where K_{enc} is used to encrypt the data and K_{mac} is used to generate the MAC. The MAC is generated using the formula:

$$MAC = HMAC(K_{mac}, CM) \tag{10}$$

During the verification process, if the data is tampered with, the MAC verification will fail, thus ensuring the integrity of the data.

6.3. Correctness analysis

The data is cryptographically sliced to generate hash labels, and then MHT and interval Merkle trees are constructed. The storage proof derivation formula is as follows:

$$R_j = H(H(T_k \| SiblingHash_1) \dots \| SiblingHash_l) \tag{11}$$

$$\begin{aligned} P &= \sum (g^{R_j} h^r \bmod p) \mathrm{Chal}_j \bmod p \\ &= \sum C_j \, \mathrm{Chal}_j \bmod p \\ &= Q \end{aligned} \tag{12}$$

Through the layer-by-layer aggregation property of the Merkle hash tree, the root hash R_j uniquely characterizes the complete dataset, and any data tampering will result in hash deviations for at least one validation path. The homomorphic nature of the Pedersen commitment guarantees that the CSP generates a consistent user-computed committed aggregation value if and only if the CSP actually holds the original data.

6.4. Immutability

This approach ensures data immutability using blockchain and TPV:The root hash of data created by the Merkle tree is recorded in the blockchain. The blockchain's immutable nature ensures that any modification to the data results in a mismatch of the root hash, rapidly discovering data tampering. In the Kademlia network, the data transmission credentials generated by each node are combined to form a TPV. The Chain tracks transmission channels and node contributions, provides a transparent tracking system, and assures that data and node behavior during transmission are tamper-resistant.

7. Application scenarios

7.1. Medical domain

Privacy protection and data integrity are the most crucial considerations for medical data processing. We use ECIES encryption to safeguard the confidentiality and privacy of patient data throughout transmission and auditing. This solution not only encrypts proof of data ownership, but it also includes a secure transport layer that protects patient data integrity from unauthorized manipulation during segmented storage and transmission. Our approach also incorporates a transmission credential chaining mechanism that records each node's behavior throughout data transmission and assures the transmission process's integrity and security by checking the data segment hash values. This approach adds an additional layer of protection to medical data transfer, making it more safe and dependable in complicated network contexts, maintaining patient data integrity from generation to transmission, and avoiding tampering.

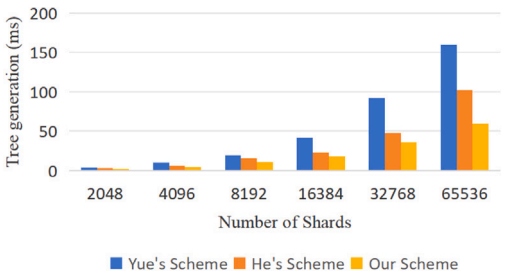


Fig. 5. Tree generation time.

7.2. Internet of things

The most significant advantage of this technology in IoT applications is its ability to efficiently handle and transmit data generated by large numbers of devices via data grouping and slicing techniques. Constructing data fragments as Merkle trees and quick localization using interval tree indexing are appropriate for IoT's distributed and large-scale data processing demands. The Kademlia method uses heteroscedastic distances to quickly discover the target data, hence optimizing data distribution among nodes and reducing storage strain on individual devices. Its adaptability enables the network to dynamically modify the data storage location based on node performance and real-time network conditions, prioritizing nodes with quick response times and enhancing data access efficiency. Simultaneously, the transmission credential chain may ensure data security during node transmission, as well as transmission integrity and traceability throughout the distributed network. IoT devices generate a wide range of data in huge volumes, and group construction and dispersed transmission significantly improve data processing efficiency.

8. Assessment and analysis

To assess the feasibility and performance of our proposed strategy, we created a prototype system on a PC with an Intel Core i5 CPU and 16 GB RAM. In Python, we develop the Kademlia method and use ECC cryptography over the secp256k1 curve + AES-256-GCM authenticated encryption.

8.1. Data integrity

First, we examine how the tree formation time increases with the number of data chunks. In the experiment, we divide the 256 data pieces in the scheme into one group. The total number of slices is estimated to be 2048, 4096, 8192, 16 384, 32 768, and 65 536. Fig. 5 demonstrates a positive correlation between slice generating time and number of slices. However, our technique outperforms He's scheme [13], and Yue's scheme [11] in terms of generation time since it decreases the number of nodes and depth required to build the tree with the same total number of slices while improving parallelism.

Second, we compare query efficiency. We set the number of query slices to 128, 256, 512, 1024, and 2048. Fig. 6 shows that both our and He's schemes are more efficient in terms of overall query time. This is because our technique uses the Interval Tree data structure, which is ideal for fast discovering and retrieving interval-based queries. Although He's Scheme [13] offers binary search, its performance is restricted by the data's organization. He's scheme can access the data directly, while our scheme needs to obtain the root hash value of

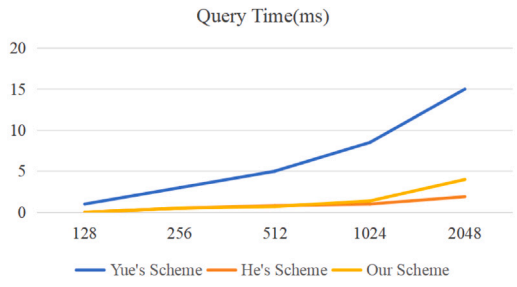


Fig. 6. Search time.

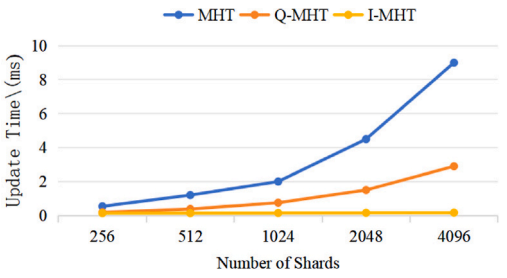


Fig. 7. Update time.

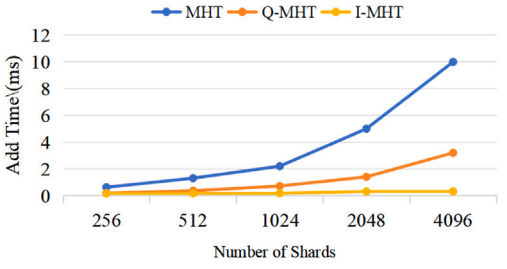


Fig. 8. Add time.

the data through the interval tree first, and then needs to traverse the corresponding hash tree in order to locate the data, which is an additional step resulting in a slight increase in query time.

Setting the number of leaf nodes to 64 to construct the MHT, the interval Merkle tree is compared with the MHT and the Quadratic Merkle Tree (Q-MHT) in the update and insert operations. The time comparisons of the add and update operations are shown in Figs. 7 and 8. It can be seen from the figures that the time consumed by this scheme is basically no more than 1 ms, no matter the join operation and the modify operation, which is due to the fact that the data blocks are constructed in groups so that as the data slices increase but the height of a single MHT remains unchanged. Secondly when inserting and updating, I-MHT performs binary search based on the interval where the index is located, which can quickly locate the new node to be inserted or updated.

Finally, we compare validation times for various numbers of challenge slices. Fig. 9 illustrates that verification time grows with the number of challenge slices. It can be seen that our time is similar to Chen's[15], which is that we use similar ideas to deal with the verification efficiency and the construction of Merkle tree, but the LP-MHT structure proposed by Chen is prone to the problems of uneven data distribution and waste of space resources. While He's scheme is higher, the bilinear pairing operation has high computational complexity, and pairing operation is required for each data block verification, which leads to an increase in the computational cost of the verification process, and the performance of the system decreases significantly, especially in the case of large-scale data verification. Meanwhile, labels

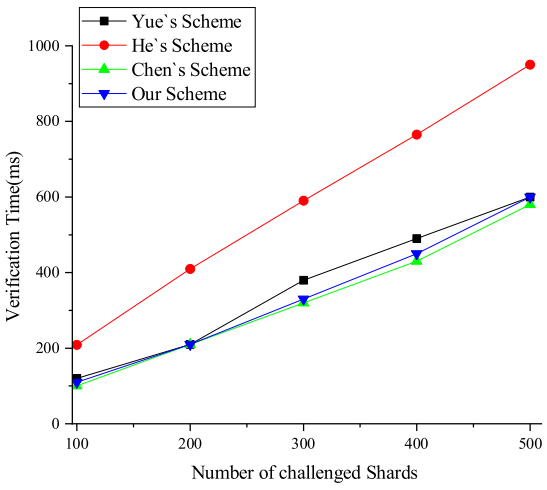


Fig. 9. Verification time.

Table 3
Comparison of Kademlia, Chord, and Pastry/Tapestry.

Metric	Kademlia	Chord	Pastry/Tapestry
RouteTable maintenance	Simple	Medium	Complexity
Update frequency	Low	High	High
Application	Large-scale dynamic networks	Small and medium-sized networks	Large-scale networks
HopCount Redundancy	Few High	Few Low	Few Moderate

and hash values need to be provided for verification, which may leak data structure or content information, according to which attackers can analyze data characteristics or access patterns, leading to data privacy leakage.

8.2. Data forward

We evaluated the Kademlia algorithm alongside Chord, Pastry, and Tapestry in Table 3. Kademlia excels in efficient data storage and retrieval for large-scale networks. It minimizes storage and communication expenses by enabling nodes to store only segments of critical data and swiftly find target data using XOR distance. The algorithm maintains network node accuracy and data access reliability through periodic routing table updates and timeout retries. It dynamically optimizes data storage locations based on node performance and network conditions, prioritizing fast-responding nodes to enhance network efficiency and stability. Fig. 10 depicts the maximum number of hops in Kademlia's routing algorithm.

Fig. 11 clearly depicts a positive association between the quantity of credentials and authentication time. When employing the Kademlia method for data transmission, we only need to select one or two nodes from the neighboring nodes to complete the transmission task, resulting in a limited number of nodes involved in each transfer. Furthermore, during the authentication process, we execute authentication by comparing the consistency of the data hash value to the hash value of the transmission credentials, which substantially decreases computing resource consumption and latency.

8.3. Scalability

In terms of storage overhead, the system slices and groups the raw data, constructs a MHT for each group, generates labeled values for

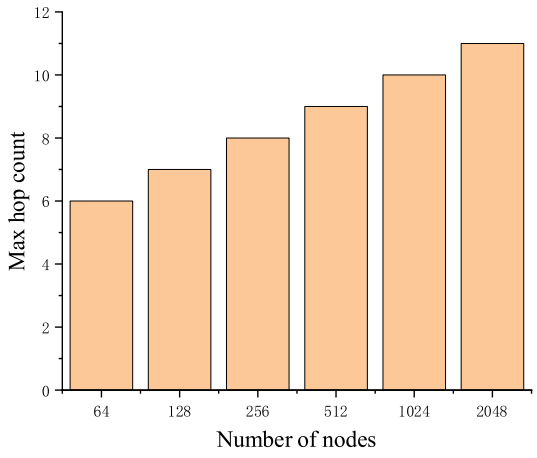


Fig. 10. Hop count.

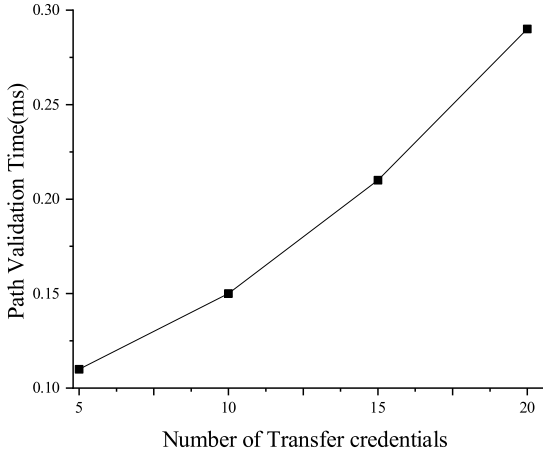


Fig. 11. Path verification time.

the root node of each tree after Pedersen commitment, and ultimately constructs an interval Merkle tree with these labels. Each data block is stored only once and the overall hash node storage grows linearly with the amount of data. If the total number of data is d and the size of each group is n, the total number of labels is approximately $t = \frac{d}{n}$. Therefore, the overall complexity of data storage is $O(ds + d + t)$, where s is the individual data block size. This structure ensures that the storage grows linearly with the data size and has good scalability.

In terms of verification overhead, the verifier needs to complete two parts of the computation: one is to compute the Merkle path and verify that the root hash agrees with the Pedersen commitment value; the other is the digital signature chain. Specifically, about log n times of hash operations are used to compute the MHT path, and one modulo power verification of Pedersen commitment (requiring 2 modulo powers and 1 multiplication mod p). In addition, each data transmission chain contains a digital signature verification with computational complexity $O(k^2)$, where k is the number of encryption bits (e.g., 256 bits for ECC). The overall verification complexity is $O(log\, n + k^3 + h\, k^2)$, where Pedersen modulo power is the main bottleneck. As a result, the system still has good validation scalability when the data volume is large, but the computational cost rises under high-strength encryption parameters.

For routing table maintenance, the scheme uses Kademlia protocol to organize the network topology. Each node maintains m routing buckets(m is the bit length of the node ID) and each bucket holds at most k nodes with a per-node maintenance cost of $O(k\, log\, N)$. During data query and transmission, the number of hops between nodes is at most $O(log\, N)$, and each hop requires a path construction with digital signature verification. The mechanism is scalable in large-scale networks as all core operations are maintained at the logarithmic level.

Overall, the storage overhead of the scheme grows linearly with the data volume, which depends on the data slicing and MHT construction strategy, and reasonable grouping can reduce the growth rate; the verification overhead is greatly affected by the tree structure and encryption computation, and it may become a bottleneck, but it can be optimized by batch verification and other methods; the maintenance of the routing table mainly depends on the ID space and the size setting of the bucket, and so on. Finding efficiency is high, and it has advantages in scalability, and it needs to optimize the update strategy to maintain performance in complex networks.

9. Conclusion

In this research, we propose a blockchain-based data storage and transmission integrity verification method that enables fast data indexing by building Merkle and interval trees and using the ECIES encryption method to assure the security of data proofs. The Kademlia algorithm is used to optimize the data backup and transmision process, and a TPV is established to record each node's transmission contribution, ensuring data integrity. In the future, we will introduce an authorization mechanism that allows transmission nodes to cache popular data to improve data retrieval efficiency, as well as build a more efficient consensus mechanism to increase data retrieval and network consensus efficiency, hence improving overall system performance.

CRediT authorship contribution statement

Zhijie Han: Writing – review & editing, Supervision, Funding acquisition, Conceptualization. **XingBo Xie:** Writing – original draft, Methodology, Investigation. **Xiaoyu Du:** Supervision, Methodology, Conceptualization. **Ying Du:** Validation, Methodology, Conceptualization. **Xin He:** Writing – review & editing, Supervision, Conceptualization.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Acknowledgments

This article is supported in part by the National Natural Science Foundation of China under projects 92367302, the Special Project for Key R&D and the Promotion of Science, Technology Department of Henan Province (242102210202, 242102210196)

Data availability

No data was used for the research described in the article.

References

[1] China internet network information center releases the 46th 'statistical report on China's internet development status', J. Natl. Libr. China 32 (2) (2023) 1.
[2] J. Huan, Design of a High Availability Storage System Based on Virtualization (Master's thesis), South China University of Technology, 2016.
[3] S. Nakamoto, Bitcoin: A peer-to-peer electronic cash system, 2008.
[4] J. Benet, Ipfs-content addressed, versioned, p2p file system, 2014, arXiv preprint arXiv:1407.3561.
[5] J. Benet, N. Greco, Filecoin: A decentralized storage network (1) (2018) 1–36.
[6] I. Storj Labs, Storj: A decentralized cloud storage network framework, 2018.
[7] V. Trón, The book of swarm-v1, 2020, 0 pre-release 7 november 17 (2020).

[8] J. Liu, K. Huang, H. Rong, H. Wang, M. Xian, Privacy-preserving public auditing for regenerating-code-based cloud storage, IEEE Trans. Inf. Forensics Secur. 10 (7) (2015) 1513–1528.
[9] J. Zhang, R. Lu, B. Wang, X.A. Wang, Comments on privacy-preserving public auditing protocol for regenerating-code-based cloud storage, IEEE Trans. Inf. Forensics Secur. 16 (2020) 1288–1289.
[10] J. Xue, C. Xu, Y. Zhang, L. Bai, Dstore: a distributed cloud storage system based on smart contracts and blockchain, in: Algorithms and Architectures for Parallel Processing: 18th International Conference, ICA3PP 2018, Guangzhou, China, November 15-17, 2018, Proceedings, Part III 18, Springer, 2018, pp. 385–401.
[11] D. Yue, R. Li, Y. Zhang, W. Tian, C. Peng, Blockchain based data integrity verification in P2P cloud storage, in: 2018 IEEE 24th International Conference on Parallel and Distributed Systems, ICPADS, IEEE, 2018, pp. 561–568.
[12] Z. Liu, L. Ren, Y. Feng, S. Wang, J. Wei, Data integrity audit scheme based on quad merkle tree and blockchain, IEEE Access 11 (2023) 59263–59273, http://dx.doi.org/10.1109/ACCESS.2023.3240066.
[13] K. He, J. Shi, C. Huang, X. Hu, Blockchain based data integrity verification for cloud storage with t-merkle tree, in: Algorithms and Architectures for Parallel Processing: 20th International Conference, ICA3PP 2020, New York City, NY, USA, October 2–4, 2020, Proceedings, Part III 20, Springer, 2020, pp. 65–80.
[14] J. Li, J. Wu, G. Jiang, T. Srikanthan, Blockchain-based public auditing for big data in cloud storage, Inf. Process. Manage. 57 (6) (2020) 102382.
[15] C. Chen, L. Wang, Y. Long, Y. Luo, K. Chen, A blockchain-based dynamic and traceable data integrity verification scheme for smart homes, J. Syst. Archit. 130 (2022) 102677.
[16] H. Wang, X.A. Wang, S. Xiao, J. Liu, Decentralized data outsourcing auditing protocol based on blockchain, J. Ambient. Intell. Humaniz. Comput. 12 (2) (2021) 2703–2714.
[17] Z. Tu, X.A. Wang, W. Du, Z. Wang, M. Lv, An improved multi-copy cloud data auditing scheme and its application, J. King Saud Univ.- Comput. Inf. Sci. 35 (3) (2023) 120–130.
[18] P. Zhang, X. Pang, N. Kumar, G.S. Aujla, H. Cao, A reliable data-transmission mechanism using blockchain in edge computing scenarios, IEEE Internet Things J. 9 (16) (2020) 14228–14236.
[19] B. Yu, X. Li, H. Zhao, T. Zhou, A scalable blockchain network model with transmission paths and neighbor node subareas, Comput. (2022) 1–25.
[20] T. Wu, G. Jourjon, K. Thilakarathna, P.L. Yeoh, MapChain-D: A distributed blockchain for IIoT data storage and communications, IEEE Trans. Ind. Inform. 19 (9) (2023) 9766–9776.
[21] M. Yuan, Y. Xu, C. Zhang, Y. Tan, Y. Wang, J. Ren, Y. Zhang, TRUCON: blockchain-based trusted data sharing with congestion control in internet of vehicles, IEEE Trans. Intell. Transp. Syst. 24 (3) (2022) 3489–3500.
[22] T. Shen, Q. Sun, C. Zhang, F. Bai, SEKad: a scalable protocol for blockchain networks with enhanced broadcast efficiency, Clust. Comput. 27 (3) (2024) 3481–3498.
[23] V. Gayoso Martínez, L. Hernández Encinas, C. Sánchez Ávila, A survey of the elliptic curve integrated encryption scheme, 2010.
[24] T.H. Cormen, C.E. Leiserson, R.L. Rivest, C. Stein, Introduction To Algorithms, MIT Press, 2022.
[25] P. Maymounkov, D. Mazieres, Kademlia: A peer-to-peer information system based on the xor metric, in: International Workshop on Peer-To-Peer Systems, Springer, 2002, pp. 53–65.



Zhijie Han received the Ph.D. degree from Soochow University. He is currently a Professor with the School of Software, Henan University. He is a CCF member. His research interests include the architecture and performance analysis of data center networks, detection of network traffic anomalies, and searchable encryption.



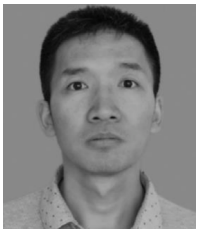
Xingbo Xie is currently pursuing a master's degree at Henan University. His research interests include Blockchain.



Xiaoyu Du received the Ph.D. degree from Nanjing University of Posts and Telecommunications. She is currently a Professor with the School of Computer and Information Engineering, Henan University. She is a CCF member. She research interests include the Internet of Things (IoT) security, the architecture and performance evaluation of data center networks, the resilience and anti-jamming studies in wireless sensor networks and vehicular networks.



Ying Du received the Ph.D. degree from Nanjing Normal University. She is currently an Associate Professor with the School of Computer and University.spatiotemporal Information Engineering, big the data, architecture Henan and performance analysis of data center networks, and software engineering.



Xin He received the Ph.D. degree from Xi'an Jiaotong University. He is currently a Professor and an Dean of the School of Software, Henan University. He is a CCF member and ACM member. His research interests include computer networks, opportunistic networks, mobile computing, and ad hoc network.